Desarrollo Multiplataforma con React Native

Día 2: Componentes, Estado y Estructura

Carlos Rodríguez Contreras
carlos.mailhub@gmail.com
karrocon

22 de abril de 2026

</> Teoría breve + práctica inmediata

Agenda del día

- Repaso Día 1
- Props y componentes
- useState
- Renderizado condicional
- Listas con map()
- Estructura de proyecto
- Ejercicios

Objetivo del día

Aprender **props**, **estado** y estructura.

Todo es nuevo hoy: teoría breve → práctica inmediata.

Horario orientativo

16:00 Repaso
16:20 Props → mini-ej
17:00 useState → mini-ej
18:00 **DESCANSO 30 min**
18:30 Condicional → mini-ej
18:55 Listas → mini-ej
19:25 Estructura → mini-ej
19:55 **PARADA: ejercicios**
20:25 Cierre 20:30 Fin

Repaso del Día 1

Lo que ya sabemos

Repaso del Día 1

16:00 — 16:20 Repaso + preguntas

Lo que ya hacemos:

```
import { StyleSheet, Text, View } from '
    react-native';

export default function App() {
  return (
    <View style={s.caja}>
      <Text style={s.titulo}>Hola Mundo</
          Text>
      <Text>Subt tulo</Text>
    </View>
  );
}
const s = StyleSheet.create({
  caja: { flex: 1, alignItems: 'center',
          justifyContent: 'center' },
  titulo: { fontSize: 28, fontWeight: 'bold'
        },
});
```

Cubierto el Día 1:

- ✔ Expo + proyecto desde cero
- ✔ `View`, `Text`, `Image`
- ✔ JSX y expresiones `{ }`
- ✔ `StyleSheet` + Flexbox

➔ Hoy empezamos

Props y componentes
Estado con `useState`
Renderizado condicional
Listas con `map()`
Estructura de proyecto

Props y componentes

Reutilización y composición

TypeScript esencial — tipos y estructuras

Tipos primitivos:

```
let nombre: string = 'Ana';
let edad: number = 20;
let activo: boolean = true;
let lista: string[] = ['a', 'b'];
```

Tipo personalizado (`type`):

```
type Usuario = {
  nombre: string;
  edad: number;
  email?: string; // opcional
};
const u: Usuario = {
  nombre: 'Ana', edad: 20
};
```

Función tipada:

```
// funcion normal:
function suma(a: number,
    b: number): number {
  return a + b;
}
// arrow function (mas comun en RN):
const doble = (x: number) => x * 2;
```

Para devs de Java:

`type` ≈ clase de datos.
No hay `private`/`public`:
todo es público por defecto.

📖 `typescriptlang.org/docs/handbook`

¿Qué son las props?

Definición del componente:

```
type SaludoProps = {
  nombre: string;
  edad: number;
  ciudad?: string; // opcional
};
function Saludo({ nombre, edad,
    ciudad = 'Granada' }: SaludoProps) {
  return (
    <View>
      <Text>{nombre}, {edad}</Text>
      <Text>{ciudad}</Text>
    </View>
  );
}
```

Uso:

```
<Saludo nombre="Ana" edad={20} />
<Saludo nombre="Carlos" edad={25}
        ciudad="Madrid" />
```

Props son:

- Parámetros del componente
- Se pasan como atributos JSX
- Solo lectura (nunca modificar)
- ? = opcional

Componente = función que **recibe props** y devuelve JSX.

✔ TypeScript garantiza que no falta ninguna prop obligatoria en compilación.

Componentes reutilizables

Componente:

```
type ItemProps = {
  texto: string;
  hecho: boolean;
};

function Item({ texto, hecho }:
    ItemProps) {
  return (
    <Text style={{
      textDecorationLine:
        hecho ? 'line-through' : 'none',
      color: hecho ? 'gray' : 'black',
    }}>
      {texto}
    </Text>
  );
}
```

Uso:

```
<Item texto="Estudiar" hecho={true}
    />
<Item texto="Practicar" hecho={false
    } />
<Item texto="Descansar" hecho={false
    } />
```

Cuándo extraer

Cuando repites el mismo bloque JSX 2+ veces, o cuando el código se vuelve difícil de leer.

Componentes de clase — el paralelo con Java

Clase (estilo Java — legacy en RN):

```
import React, { Component } from 'react';
import { Text, View } from 'react-native';
type Props = { nombre: string };
type State = { n: number };

class Saludo extends
    Component<Props, State> {
  state = { n: 0 };
  sumar = () => this.setState({
    n: this.state.n + 1
  });
  render() {
    return (
      <View>
        <Text>{this.props.nombre}</Text>
        <Text>{this.state.n}</Text>
      </View>
    );
  }
}
```

Equivalente funcional:

```
function Saludo(
    { nombre }: Props) {
  const [n, setN] = useState(0);
  return (
    <View>
      <Text>{nombre}</Text>
      <Text>{n}</Text>
    </View>
  );
}
```

¿Cuál usar?

Usa funcionales.

Las clases son legacy desde 2019.

Los Hooks **solo funcionan** en componentes funcionales.

Mini-ejercicio — Props

</> Mini-ejercicio (10 min)

Crea un componente `Tarjeta` con las props:

`nombre: string` `ciudad: string` `emoji?: string`

- Define el tipo `TarjetaProps`
- Muestra nombre y ciudad centrados con estilos
- Si existe `emoji`, muéstralo al lado del nombre
- Úsala en `App.tsx` con tus datos reales

Bonus: añade una prop `color?: string` para el fondo de la tarjeta (default: `'#eef'`).

useState en profundidad

Estado, eventos e inputs controlados

¿Qué es un Hook?

Definición:

Una función de React que empieza por `use` y añade funcionalidades del framework a un componente funcional.

Hooks principales:

- `useState` — estado local ← hoy
- `useEffect` — efectos / API (Día 4)
- `useCallback` — memorizar funciones
- `useMemo` — valores memo (Día 7)
- `useContext` — estado compartido

📖 `react.dev/reference/react/hooks`

⚠ Reglas de los Hooks

1. Solo en componentes **funcionales**
2. Siempre al nivel superior

✖ dentro de `if` o `for`

3. El nombre empieza por `use`

ℹ Para devs de Java

Acceso al estado sin `this`.

`useState(0)` ≈

```
private int n = 0;
```

Múltiples estados

```
const [nombre, setNombre]
  = useState('');
const [edad, setEdad]
  = useState(0);
const [activo, setActivo]
  = useState(false);

// Cada uno es independiente:
setNombre('Ana');
// edad y activo no cambian

// Alternar booleano:
setActivo(!activo);
```

Reglas del estado:

- Tantos `useState` como necesites
- Cada uno es independiente
- Cambiar uno **no afecta** a los demás

⚠ El estado es **local** al componente. Los hermanos no lo comparten.

Inputs controlados

```
const [texto, setTexto]
  = useState('');

<TextInput
  value={texto}              // lo que ves
  onChangeText={setTexto} // actualiza
  placeholder="Escribe..."
  style={styles.input}
/>

<Text>Escribiste: {texto}</Text>
<Button
  title="Limpiar"
  onPress={() => setTexto('')}
/>
```

¿Por qué "controlado"?

El estado **controla** el valor del input. Siempre sabes exactamente qué hay en el campo.

Props útiles de TextInput:

- `keyboardType="numeric"`
- `secureTextEntry` (contraseñas)
- `maxLength={50}`
- `multiline`

Estado con objetos y arrays

Objeto — spread obligatorio:

```
const [form, setForm] = useState({
  nombre: '', email: ''
});

// Actualizar un campo:
setForm({ ...form, nombre: 'Ana' });
```

Array — siempre crea uno nuevo:

```
const [lista, setLista]
  = useState<string[]>([]);

const anadir = (item: string) =>
  setLista([...lista, item]);

const eliminar = (i: number) =>
  setLista(lista.filter(
    (_, idx) => idx !== i));
```

⚠ Inmutabilidad

✗ `form.nombre = 'x'`

✓ `{...form, nombre: 'x'}`

✗ `lista.push('x')`

✓ `[...lista, 'x']`

React necesita un nuevo objeto/array para detectar el cambio y re-renderizar.

Mini-ejercicio — useState

</> Mini-ejercicio (15 min)

Crea tu primer contador con estado:

- Muestra un número en pantalla con `useState(0)`
- Botón **+1** que lo incrementa
- Botón **-1** que lo decrementa (mínimo 0)
- Botón **Reset** que lo vuelve a 0
- Añade un `TextInput` y muestra «¡Hola {nombre}!»

Bonus: extrae la lógica del contador a un custom hook `useContador`.

Pista: `setN(Math.max(0, n - 1))` `setNombre('')`

Descanso — 30 minutos

18:00 — 18:30 Descanso

30 minutos

Al volver: renderizado condicional y listas

Renderizado condicional

Mostrar cosas según el estado

Ternario, && y variable

Ternario (A o B):

```
{activo
  ? <Text>Activo     </Text>
  : <Text>Inactivo</Text>
}
```

&& (mostrar o nada):

```
{error && (
  <Text style={s.error}>
    {error}
  </Text>
)}
```

Variable con if (lógica compleja):

```
let badge = null;
if (n === 0) badge = <Text>       </Text>;
else if (n > 5) badge = <Text>       </Text>;
return <View>{badge}<Text>{n}</Text></View>;
```

Cuándo usar cada uno

Ternario → dos opciones (A o B)

&& → mostrar o no mostrar

Variable if → tres o más casos

⚠ `0 && ...` renderiza el cero.
Usa `!!n && ...` o un ternario.

`<>...</>` agrupa sin añadir un `View` extra.

Mini-ejercicio — Renderizado condicional

</> Mini-ejercicio (10 min)

Mejora el contador que acabas de hacer:

- Si `n === 0`: número en gris y botón -1 deshabilitado (`disabled`)
- Si `n > 5`: número en rojo con emoji
- Si `nombre` vacío: mostrar *«Escribe tu nombre»* en cursiva

Bonus: muestra mensajes distintos para `n === 0`, `n < 5` y `n >= 5`.

Pista disabled: `<Button ... disabled={n === 0} />`

Pista estilo dinámico: `style={{ color: n > 5 ? 'red' : 'black' }}`

Listas con map()

Renderizar colecciones de datos

Array.map() — La clave

```
type Ciudad = {
  id: number; nombre: string; pais: string;
};
const ciudades: Ciudad[] = [
  { id: 1, nombre: 'Madrid', pais: 'Espa a'
      },
  { id: 2, nombre: 'Par s',  pais: 'Francia'
      },
];

export default function App() {
  return (
    <View style={{ padding: 16 }}>
      {ciudades.map((c) => (
        <Text key={c.id}>
          {c.nombre}     {c.pais}
        </Text>
      ))}
    </View>
  );
}
```

Reglas:

- **key** único y estable
- Usa el `id`, nunca el índice
- Devuelve JSX por cada elemento

⚠ Sin `key` → warning.
`key={index}` → bugs en listas que cambian de orden.

`Ciudad[]` garantiza que cada objeto tiene `id`, `nombre` y `pais`.

Lista con estado — añadir y eliminar

```
const [items, setItems]
  = useState<string[]>([]);
const [texto, setTexto]
  = useState('');

const anadir = () => {
  if (!texto.trim()) return;
  setItems([...items, texto]);
  setTexto('');
};

const eliminar = (i: number) => {
  setItems(
    items.filter((_, idx) => idx !== i)
  );
};
```

Inmutabilidad

- ✗ `items.push(...)`
- ✓ `[...items, nuevo]`
- ✗ `items.splice(...)`
- ✓ `items.filter(...)`

Crea siempre un **nuevo array** para que React detecte el cambio y re-renderice.

Mini-ejercicio — Listas

</> Mini-ejercicio (15 min)

Crea una lista de ciudades favoritas:

- Empieza con 3 ciudades hardcodeadas en el estado
- Muéstralas con `map()` y `key`
- Añade `TextInput` + botón «Añadir ciudad»
- Cada ciudad tiene un botón «» para eliminarla

Bonus: muestra el total encima de la lista: «{n} ciudades».

Pista: `<Text>Total: {ciudades.length}</Text>`

Estructura de proyecto

blank-typescript y organización real

La plantilla blank-typescript

Crear el proyecto:

```
npx create-expo-app miApp
-template blank-typescript
```

Estructura recomendada:

```
miApp/
  App.tsx
  src/
    components/  UI reutilizable
    screens/  cada pantalla
    hooks/  lógica con estado
    utils/  funciones puras
    types/  interfaces y tipos
    constants/  colores, textos
  assets/   __tests__/
```

Regla de oro

components/ → UI reutilizable
screens/ → cada pantalla completa
hooks/ → lógica con `useState`
utils/ → funciones puras sin estado
types/ → interfaces y tipos TS
constants/ → colores, textos fijos

Para empezar basta con `components/` y `screens/`. Añade el resto cuando lo necesites.

¿Dónde va cada cosa? — Ejemplos

components/Boton.tsx

```
type BotonProps = {
  titulo: string;
  onPress: () => void;
};

export function Boton(
    { titulo, onPress }: BotonProps) {
  return (
    <TouchableOpacity onPress={onPress}>
      <Text>{titulo}</Text>
    </TouchableOpacity>
  );
}
```

hooks/useContador.ts

```
export function useContador(ini = 0) {
  const [n, setN] = useState(ini);
  return {
    n,
    inc: () => setN(n + 1),
    dec: () => setN(Math.max(0, n-1)),
    reset: () => setN(0),
  };
}
```

constants/colors.ts

```
export const Colors = {
  primario: '#2078F3',
  fondo: '#F5F5F5',
  error: '#E53935',
};
```

Mini-ejercicio — Clasifica los ficheros

</> Mini-ejercicio (10 min)

¿En qué carpeta pondrías cada uno de estos ficheros?

Fichero	Carpeta
`useListaTareas.ts`	_____/
`PantallaInicio.tsx`	_____/
`COLORS.ts`	_____/
`BotonVerde.tsx`	_____/
`formatFecha.ts`	_____/
`Tarea.ts` (solo tipo)	_____/

Respuestas: `hooks/` | `screens/` | `constants/` | `components/` | `utils/` | `types/`

Ejercicios en clase

Tres retos para cerrar el día

Ejercicio 1 — Contador completo

Enunciado

Contador con **+1**, **-1** y **Reset**. No baja de 0.
El número se muestra en rojo si es mayor que 5.
Bonus: extrae la lógica a un custom hook `useContador`.

Pista:

```
const [n, setN] = useState(0);
const inc   = () => setN(n + 1);
const dec   = () => setN(Math.max(0, n - 1));
const reset = () => setN(0);
```

`dia-02/ejercicios/ej01-contador/Solucion.js`

Ejercicio 2 — Formulario de registro

Enunciado

Formulario: nombre, email y botón «Registrar».

Valida que no estén vacíos.

Si OK → mensaje de bienvenida.
Si falta algo → error en pantalla.

Bonus: valida formato email (`includes('@')`).

ej02-formulario/Solucion.js

Pista:

```
const [nombre, setNombre] = useState('');
const [email, setEmail]   = useState('');
const [error, setError]   = useState('');
const [ok, setOk]         = useState(false
    );

const registrar = () => {
  if (!nombre || !email) {
    setError('Rellena todos los campos');
    return;
  }
  setOk(true);
};
```

Ejercicio 3 — Lista de tareas

Enunciado

To-do list completa:

- ✔ Input + botón «Añadir»
- ✔ Lista con `map()` y `key`
- ✔ Botón «×» en cada tarea

Bonus: tocar la tarea la marca como completada (tachado).

`ej03-lista-tareas/Solucion.js`

Pista:

```
const [texto, setTexto] = useState('');
const [tareas, setTareas] = useState([])
    ;

const anadir = () => {
  if (!texto.trim()) return;
  setTareas([...tareas,
    { id: Date.now(), texto }]);
  setTexto('');
};

const eliminar = (id) =>
  setTareas(
    tareas.filter(t => t.id !== id));
```

Parada: Ejercicios en clase

19:55 — 20:25 Trabajo autónomo guiado

Los alumnos trabajan en los 3 ejercicios.

El profesor:

- Rota por los puestos
- Ayuda con errores de estado
- Muestra solución en proyector si hay bloqueo general

Ejercicios en: `dia-02/ejercicios/`

Criterio de éxito

- Contador: +1/-1/reset + color
- Formulario: valida + bienvenida
- Lista: añadir + eliminar

Bonus: completadas + custom hook

Resumen del día

Hoy hemos aprendido:

- ✔ Componentes con **props** tipadas
- ✔ **useState**: primer uso, objetos y arrays
- ✔ Inputs controlados (`TextInput`)
- ✔ Renderizado condicional (`? :`, `&&`)
- ✔ Listas con `map()` y `key`
- ✔ Estructura de proyecto TypeScript
- ✔ Custom hooks (bonus)

Día 3 — Próxima sesión

- **Navegación** entre pantallas
- React Navigation
- Stack Navigator
- Pasar datos entre pantallas

➜ Construiremos una app con 3 pantallas

Recursos y referencias

Documentación oficial:

- `reactnative.dev` — docs RN
- `react.dev/learn` — tutorial React
- `docs.expo.dev` — Expo SDK
- `typescriptlang.org` — TS Handbook

Playground online:

- `snack.expo.dev` — RN en el navegador
- `typescriptlang.org/play` — TS online

Para profundizar:

- `reactnative.express` — guía interactiva
- `reactnavigation.org` — nav. (Día 3)
- Expo Tutorial — paso a paso

Usa **Snack** para probar código sin instalar nada. Comparte con una URL.

`karrocon/taller-react-native-fp`
Ejercicios y soluciones del curso

¿Preguntas?

Antes de salir:

✔ Entiendes props vs estado
✔ Sabes usar `map()` con `key`
✔ Tienes clara la estructura de carpetas

Si no es así → **ahora es el momento de preguntar**